TOUGH JOIN EXAMPLES

🧩 **Tables Used**

Table: customers

customer_id	name	city
1	Akshay	Mumbai
2	Priya	Delhi
3	Raj	Pune
4	Neha	Mumbai
5	Karan	Chennai

Table: orders

order_id	customer_id	amount	order_date
101	1	5000	2025-10-01
102	2	2500	2025-10-02
103	3	4000	2025-10-03
104	1	3000	2025-10-04
105	6	2000	2025-10-05

🧱 1️⃣ **INNER JOIN — Matching rows from both tables**

```
select c.customer_id,c.name,o.order_id,o.amount
from customers c
inner join orders o
on c.customer_id=o.customer_id;
```

✅ **Output**

customer_id	name	order_id	amount
1	Akshay	101	5000
1	Akshay	104	3000
2	Priya	102	2500
3	Raj	103	4000

Explanation:
Only customers who have placed at least one order are shown. Customer 5 (Karan) and order 105 (customer_id 6 — non-existent) are excluded.

🧱 2️⃣ **LEFT JOIN — All customers, even if no order**

```
select c.customer_id,c.name,o.order_id,o.amount
from customers c
left join orders o
on c.customer_id=o.customer_id;
```

✅ **Output**

customer_id	name	order_id	amount
1	Akshay	101	5000
1	Akshay	104	3000
2	Priya	102	2500
3	Raj	103	4000
4	Neha	NULL	NULL
5	Karan	NULL	NULL

Explanation:
Shows all customers — including those without orders (Neha, Karan).

🧱 3️⃣ **RIGHT JOIN — All orders, even if customer missing**

```
select c.customer_id,c.name,o.order_id,o.amount
from customers c
right join orders o
on c.customer_id=o.customer_id;
```

✅ **Output**

customer_id	name	order_id	amount
1	Akshay	101	5000
1	Akshay	104	3000
2	Priya	102	2500
3	Raj	103	4000
NULL	NULL	105	2000

Explanation:
Order 105 has customer_id = 6 not present in customers → shows as NULL.

🧱 4️⃣ **FULL OUTER JOIN — Combine all rows from both tables**

```
select c.customer_id,c.name,o.order_id,o.amount
from customers c
full outer join orders o
on c.customer_id=o.customer_id;
```

✅ **Output**

customer_id	name	order_id	amount
1	Akshay	101	5000
1	Akshay	104	3000
2	Priya	102	2500
3	Raj	103	4000
4	Neha	NULL	NULL
5	Karan	NULL	NULL
NULL	NULL	105	2000

Explanation:
Includes all data from both tables — unmatched rows from either side appear with NULLs.

🧱 5️⃣ **SELF JOIN — Compare within same table**

Find pairs of customers from the same city

```
select a.name as customer1,b.name as
customer2,a.city
from customers a
join customers b
on a.city=b.city
where a.customer_id<b.customer_id;
```

✅ **Output**

customer1	customer2	city
Akshay	Neha	Mumbai

Explanation:
Self-join matches customers living in the same city.

🧱 6️⃣ **CROSS JOIN — Cartesian product (all combinations)**

```
select c.name,o.order_id
from customers c
cross join orders o;
```

✅ **Output (partial)**

name	order_id
Akshay	101
Akshay	102
Akshay	103
Akshay	104
Akshay	105
Priya	101
...	...

Explanation:
Every customer is paired with every order — total = 5 customers × 5 orders = **25 rows**.

💪 7️⃣ **TOUGH JOIN #1 — Find customers with more than one order**

```
select c.name,count(o.order_id) as total_orders
from customers c
join orders o
on c.customer_id=o.customer_id
group by c.name
having count(o.order_id)>1;
```

✅ **Output**

name	total_orders
Akshay	2

Explanation:
Only **Akshay** placed more than one order (101 & 104).

💪 8️⃣ **TOUGH JOIN #2 — Find customers who have not placed any order**

```
select c.name
from customers c
left join orders o
on c.customer_id=o.customer_id
where o.order_id is null;
```

✅ **Output**

name
Neha
Karan

Explanation:
Using LEFT JOIN with WHERE o.order_id IS NULL filters customers with no matching orders.

💪 9️⃣ **TOUGH JOIN #3 — Find total sales per city**

```
select c.city,sum(o.amount) as total_sales
from customers c
join orders o
on c.customer_id=o.customer_id
group by c.city;
```

✅ **Output**

city	total_sales
Mumbai	8000
Delhi	2500
Pune	4000

Explanation:
Aggregates order amounts grouped by the customer's city.

💪 🔟 **TOUGH JOIN #4 — Conditional Join (Join + Filter inside condition)**

Find customers who ordered more than ₹3000

```
select distinct c.name,o.amount
from customers c
join orders o
on c.customer_id=o.customer_id
and o.amount>3000;
```

✅ **Output**

name	amount
Akshay	5000
Raj	4000

Explanation:
AND o.amount>3000 inside the join condition filters matching rows.

💪 1️⃣1️⃣ **TOUGH JOIN #5 — Find customers from Mumbai who placed any order**

```
select c.name,o.order_id,o.amount
from customers c
inner join orders o
on c.customer_id=o.customer_id
where c.city='Mumbai';
```

✅ **Output**

name	order_id	amount
Akshay	101	5000
Akshay	104	3000

💪 1️⃣2️⃣ **TOUGH JOIN #6 — Join with Subquery (max amount per customer)**

```
select c.name,o.amount
from customers c
```

```
join orders o
on c.customer_id=o.customer_id
where o.amount=(select max(o2.amount)
        from orders o2
        where o2.customer_id=c.customer_id);
```

✅ **Output**

name	amount
Akshay	5000
Priya	2500
Raj	4000

Explanation:
Subquery finds the **maximum order amount per customer**, and join ensures we get that specific order.

🗓️ **Sample Tables**

customers

customer_id	name	city
1	Akshay	Mumbai
2	Priya	Delhi
3	Raj	Pune
4	Neha	Chennai

---

orders

order_id	customer_id	amount	order_date
101	1	5000	2025-09-25
102	2	2500	2025-10-01
103	3	4500	2025-10-03
104	1	3000	2025-10-05
105	4	1500	2025-09-20

⚙️ 1️⃣ **JOIN with DATE RANGE — Orders in October 2025**

```
select c.name,o.order_id,o.amount,o.order_date
from customers c
join orders o
on c.customer_id=o.customer_id
where o.order_date between '2025-10-01' and '2025-10-31';
```

✅ **Output**

name	order_id	amount	order_date
Priya	102	2500	2025-10-01
Raj	103	4500	2025-10-03
Akshay	104	3000	2025-10-05

Explanation:
BETWEEN selects only the records whose date is within October 2025.

⚙️ 2️⃣ **JOIN with DATE FUNCTION — Extract Month and Group Sales**

```
select extract(month from o.order_date) as
month,sum(o.amount) as total_sales
from customers c
join orders o
on c.customer_id=o.customer_id
group by extract(month from o.order_date)
order by month;
```

✅ **Output**

month	total_sales
9	6500
10	10000

Explanation:

- Month 9 → September orders (101 + 105)
- Month 10 → October orders (102 + 103 + 104)

⚙️ 3️⃣ **JOIN to Find Most Recent Order per Customer**

```
select c.name,o.order_date,o.amount
from customers c
join orders o
on c.customer_id=o.customer_id
where o.order_date=(select max(o2.order_date)
        from orders o2
        where o2.customer_id=c.customer_id);
```

✅ **Output**

name	order_date	amount
Akshay	2025-10-05	3000
Priya	2025-10-01	2500
Raj	2025-10-03	4500
Neha	2025-09-20	1500

Explanation:
Subquery finds each customer's **latest order date**.

⚙️ 4️⃣ **JOIN with DATE DIFFERENCE — Days since last order**

```
select c.name,
    round(sysdate - max(o.order_date)) as
days_since_last_order
from customers c
join orders o
on c.customer_id=o.customer_id
group by c.name;
```

✅ **Output (Assume sysdate = 2025-10-09)**

name	days_since_last_order
Akshay	4
Priya	8
Raj	6
Neha	19

Explanation:
sysdate - order_date gives the **number of days since the last order** for each customer.

⚙️ 5️⃣ **JOIN with DATE PART — Find Orders in the Last 10 Days**

```
select c.name,o.order_id,o.order_date,o.amount
from customers c
join orders o
on c.customer_id=o.customer_id
where o.order_date>=sysdate-10;
```

✅ **Output (If sysdate=2025-10-09)**

name	order_id	order_date	amount
Priya	102	2025-10-01	2500
Raj	103	2025-10-03	4500
Akshay	104	2025-10-05	3000

Explanation:
Shows only orders placed in the last 10 days.

⚙️ 6️⃣ **Conditional DATE JOIN — Join orders by month and year**

```
select c.name,o.order_id,o.amount,o.order_date
from customers c
join orders o
on c.customer_id=o.customer_id
and extract(month from o.order_date)=10
and extract(year from o.order_date)=2025;
```

✅ **Output**

name	order_id	amount	order_date
Priya	102	2500	2025-10-01
Raj	103	4500	2025-10-03
Akshay	104	3000	2025-10-05

⚙️ 7️⃣ **Tough One — Find customers who placed no order in the last 15 days**

```
select c.name
from customers c
left join orders o
on c.customer_id=o.customer_id
group by c.name
having max(o.order_date)<sysdate-15 or
max(o.order_date) is null;
```

✅ **Output (Assume sysdate=2025-10-09)**

name
Neha

Explanation:
Neha's last order was on 2025-09-20 → older than 15 days ago.

⚙️ 8️⃣ **Join + Date Filtering + Aggregate — Find total October sales per city**

```
select c.city,sum(o.amount) as october_sales
from customers c
join orders o
on c.customer_id=o.customer_id
where to_char(o.order_date,'MM-YYYY')='10-2025'
group by c.city;
```

✅ **Output**

city	october_sales
Mumbai	3000
Delhi	2500
Pune	4500

⚙️ 9️⃣ **Join + Date Comparison Between Tables**

(Example: if you had a shipments table and wanted orders shipped within 2 days — but here we simulate using same table)
Let's assume we want to **find customers who ordered before Akshay's last order date**.

```
select c.name,o.order_date
from customers c
join orders o
on c.customer_id=o.customer_id
where o.order_date<(select max(order_date)
        from orders
        where customer_id=1);
```

✅ **Output**

name	order_date
Priya	2025-10-01
Raj	2025-10-03
Neha	2025-09-20

✅ **Key Takeaways**

- Use BETWEEN, EXTRACT, TO_CHAR, and SYSDATE for date logic.
- Always format dates in 'YYYY-MM-DD' format for comparisons.
- Use aggregate functions like MAX(order_date) with GROUP BY to get last order per customer.
- Combine JOIN + HAVING for advanced date-based analysis.

---

3 Table Join Examples

Tables

1️⃣ **customers**

customer_id	name	city
1	Akshay	Mumbai
2	Priya	Delhi
3	Raj	Pune
4	Neha	Chennai

2️⃣ **orders**

order_id	customer_id	amount	order_date
101	1	5000	2025-09-25

order_id	customer_id	amount	order_date
102	2	2500	2025-10-01
103	3	4500	2025-10-03
104	1	3000	2025-10-05
105	4	1500	2025-09-20

---

3 **payments**

payment_id	order_id	payment_date	payment_mode
201	101	2025-09-26	Credit Card
202	102	2025-10-02	Cash
203	103	2025-10-04	Debit Card
204	104	2025-10-06	Credit Card
205	106	2025-10-07	Cash

---

1 **INNER JOIN across 3 tables — Get customer, order, and payment info**

```
select c.name, o.order_id, o.amount, p.payment_date,
p.payment_mode
from customers c
join orders o
on c.customer_id=o.customer_id
join payments p
on o.order_id=p.order_id;
```

✅ **Output**

name	order_id	amount	payment_date	payment_mode
Akshay	101	5000	2025-09-26	Credit Card
Priya	102	2500	2025-10-02	Cash
Raj	103	4500	2025-10-04	Debit Card
Akshay	104	3000	2025-10-06	Credit Card

Explanation:

- Only orders with **matching payment records** are shown.
- Payment 205 (order_id=106) is ignored because order 106 doesn't exist in orders.

---

2 **LEFT JOIN across 3 tables — Include all orders even if payment missing**

```
select c.name, o.order_id, o.amount, p.payment_date,
p.payment_mode
from customers c
join orders o
on c.customer_id=o.customer_id
left join payments p
on o.order_id=p.order_id;
```

✅ **Output**

name	order_id	amount	payment_date	payment_mode
Akshay	101	5000	2025-09-26	Credit Card
Akshay	104	3000	2025-10-06	Credit Card
Priya	102	2500	2025-10-02	Cash
Raj	103	4500	2025-10-04	Debit Card
Neha	105	1500	NULL	NULL

Explanation:

- Neha's order 105 has **no payment**, but still appears due to LEFT JOIN.

---

3 **Aggregate JOIN across 3 tables — Total paid per customer**

```
select c.name, sum(o.amount) as
total_order_amount, sum(p.amount) as
total_paid_amount
from customers c
join orders o
on c.customer_id=o.customer_id
left join payments p
on o.order_id=p.order_id
group by c.name;
```

✅ **Output (Assume payments.amount same as orders.amount)**

name	total_order_amount	total_paid_amount
Akshay	8000	8000
Priya	2500	2500
Raj	4500	4500
Neha	1500	NULL

Explanation:

- Aggregate functions summarize **amounts** across orders/payments.
- NULL for Neha indicates no payments.

---

4 **DATE-based 3-table join — Orders paid within last 5 days**

```
select c.name, o.order_id, o.amount, p.payment_date
from customers c
join orders o
on c.customer_id=o.customer_id
join payments p
on o.order_id=p.order_id
where p.payment_date >= sysdate-5;
```

✅ **Output (Assume sysdate = 2025-10-09)**

name	order_id	amount	payment_date
Akshay	104	3000	2025-10-06

Explanation:

- Filters **payments made in the last 5 days** using sysdate-5.

---

5 **Conditional Join (Tough) — Customers who paid with Credit Card and amount > 4000**

```
select c.name, o.order_id, o.amount,
p.payment_mode
from customers c
join orders o
on c.customer_id=o.customer_id
join payments p
on o.order_id=p.order_id
where p.payment_mode='Credit Card' and
o.amount>4000;
```

✅ **Output**

name	order_id	amount	payment_mode
Akshay	101	5000	Credit Card

Explanation:

- Combines **join + conditional filtering** on both tables.

---

6 **Self Join + 3-table Join (Advanced) — Customers from same city, their orders, and payments**

```
select c1.name as customer1, c2.name as customer2,
o.order_id, p.payment_mode
from customers c1
join customers c2
on c1.city=c2.city and c1.customer_id<c2.customer_id
join orders o
on c1.customer_id=o.customer_id
join payments p
on o.order_id=p.order_id;
```

✅ **Output**

customer1	customer2	order_id	payment_mode
Akshay	(if another Mumbai customer exists...)	...	...

Explanation:

- Combines **self join on customers** + **orders/payments**.
- Useful in cases like comparing **customers in same city who made payments**.

---

1 **Find customers who never paid with Cash but have orders**

```
select distinct c.name
from customers c
join orders o
on c.customer_id=o.customer_id
left join payments p
on o.order_id=p.order_id and
p.payment_mode='Cash'
where p.payment_id is null;
```

✅ **Output**

name
Akshay
Raj
Neha

Explanation:

- LEFT JOIN + condition p.payment_mode='Cash' → filters out customers with cash payments.
- p.payment_id IS NULL ensures we select **only those who never paid by Cash**.

---

2 **Total payments per city in October 2025**

```
select c.city, sum(o.amount) as total_order_amount,
sum(p.amount) as total_paid
from customers c
join orders o
on c.customer_id=o.customer_id
join payments p
on o.order_id=p.order_id
where extract(month from o.order_date)=10
and extract(year from o.order_date)=2025
group by c.city;
```

✅ **Output**

city	total_order_amount	total_paid
Delhi	2500	2500
Pune	4500	4500
Mumbai	3000	3000

Explanation:

- Filters **orders in October 2025**.
- Aggregates by **city**, combining **orders and payments**.

---

3 **Find customers with orders exceeding their average order amount**

```
select c.name, o.order_id, o.amount
from customers c
join orders o
on c.customer_id=o.customer_id
where o.amount > (
  select avg(amount)
  from orders
  where customer_id=c.customer_id
);
```

✅ **Output**

name	order_id	amount
Akshay	101	5000

Explanation:

- Subquery calculates **average order per customer**.
- Filters **orders above that customer's average**.

---

4 **Find the last payment per customer**

```
select c.name, o.order_id, p.payment_date,
p.payment_mode
from customers c
join orders o
on c.customer_id=o.customer_id
join payments p
on o.order_id=p.order_id
where p.payment_date = (
   select max(p2.payment_date)
   from orders o2
   join payments p2
   on o2.order_id=p2.order_id
   where o2.customer_id=c.customer_id
);
```

✅ **Output**

name	order_id	payment_date	payment_mode
Akshay	104	2025-10-06	Credit Card
Priya	102	2025-10-02	Cash
Raj	103	2025-10-04	Debit Card
Neha	105	NULL	NULL

Explanation:

- Subquery finds **latest payment per customer**.
- Joins with orders and payments to get full details.

5️⃣ **Find customers who paid more than 4000 in any order and sort by city**

```
select c.name, c.city, o.order_id, o.amount,
p.payment_mode
from customers c
join orders o
on c.customer_id=o.customer_id
join payments p
on o.order_id=p.order_id
where o.amount>4000
order by c.city, o.amount desc;
```

✅ **Output**

name	city	order_id	amount	payment_mode
Akshay	Mumbai	101	5000	Credit Card
Raj	Pune	103	4500	Debit Card

Explanation:

- Combines **amount filter + joins + ordering by city + descending order**.
- Useful for reporting high-value customers per city.
